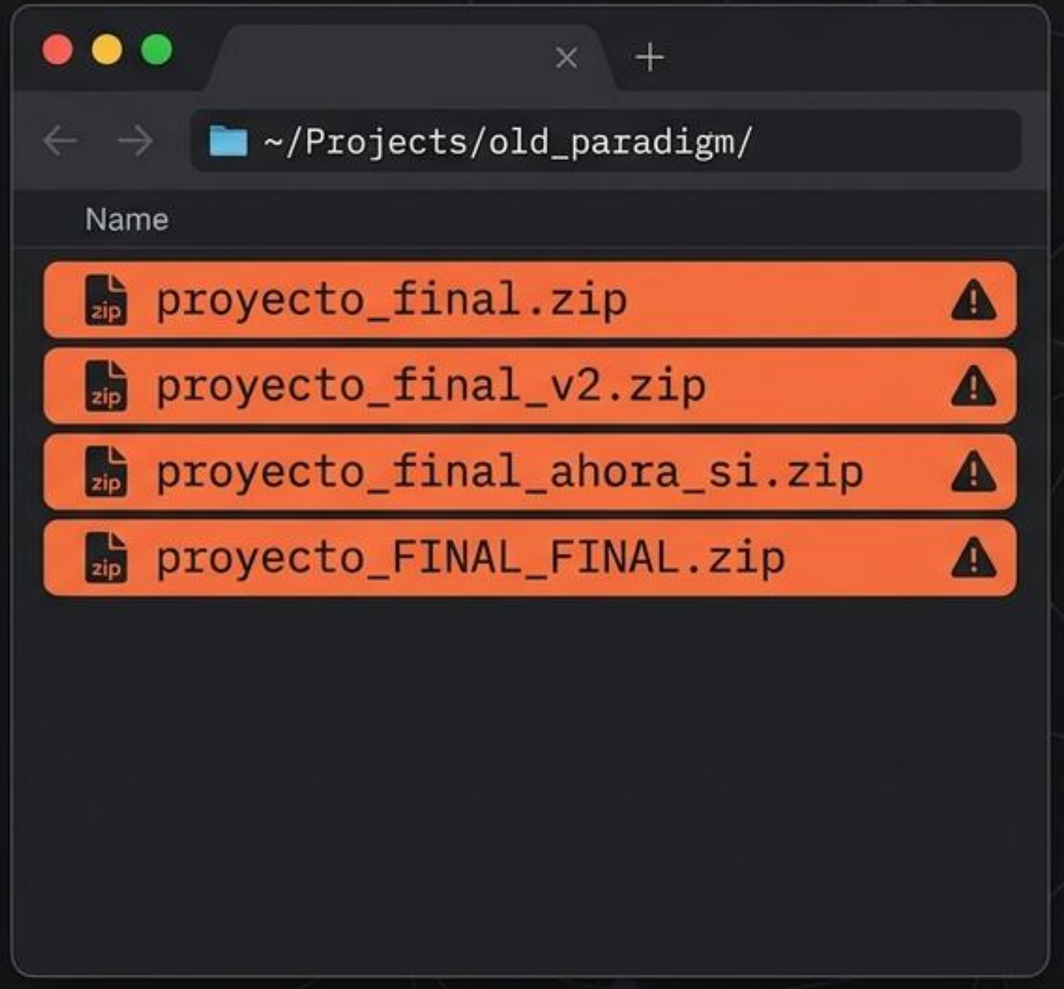

Universidad de la Amazonia
OFICINA DE
ACREDITACIÓN Y REGISTRO
CALIFICADO
INSTITUCIÓN
ACREDITADA
EN ALTA CALIDAD
Resolución No. 016136 del 17 de junio de 2026
git
Git Y Git Hub
By Equipo
Universidad de la Amazonia
TucanGo
GitHub
Spizaetus isidori

El Paradigma del Caos: El versionado manual

- Propenso a errores: **Sobrescritura** accidental de `código funcional`.
- No escalable: **Imposible colaborar simultáneamente** en el `mismo archivo`.
- **Trazabilidad nula**: ¿Quién cambió la `línea 54`, por qué lo hizo y cuándo?



Gemini Notebook



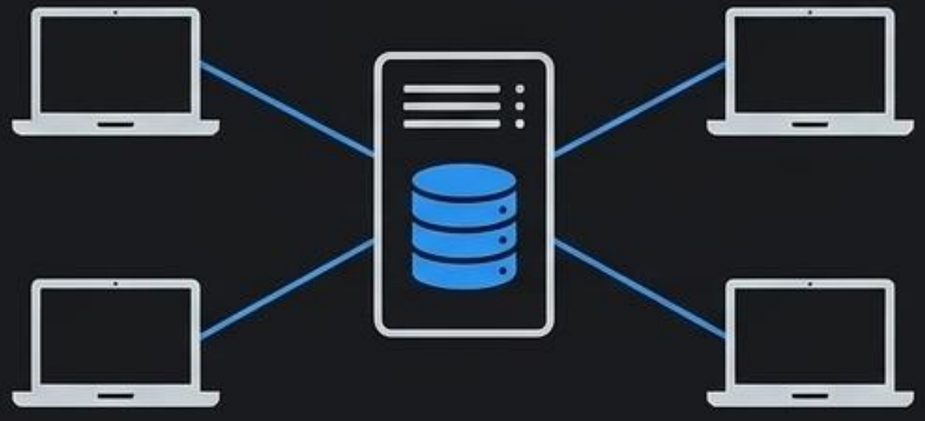
LINUS TORVALDS

(28 de diciembre de 1969 – presente)

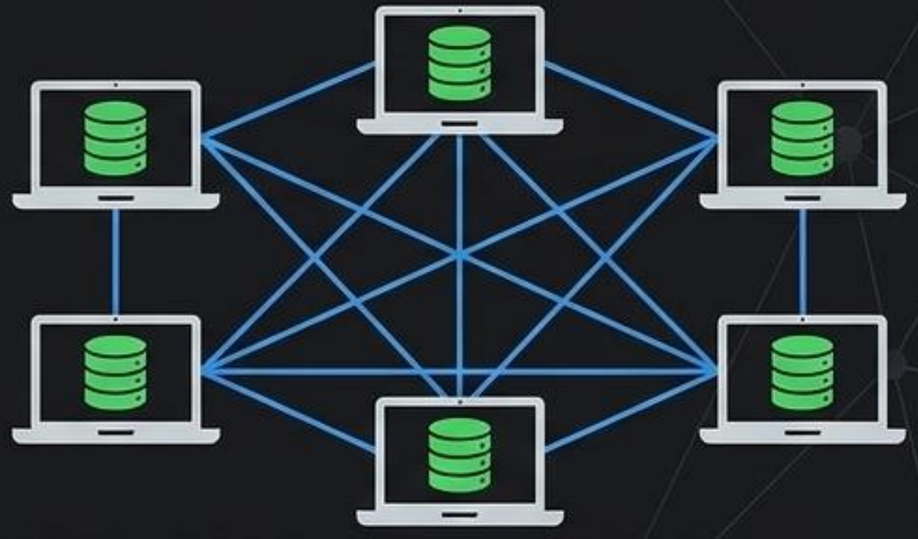
La Solución: Control de Versiones Distribuido (DVCS)

Creado por Linus Torvalds en 2005. Git no guarda archivos, guarda **instantáneas inmutables de todo el proyecto**.

Sistema Centralizado
(Punto único de fallo)

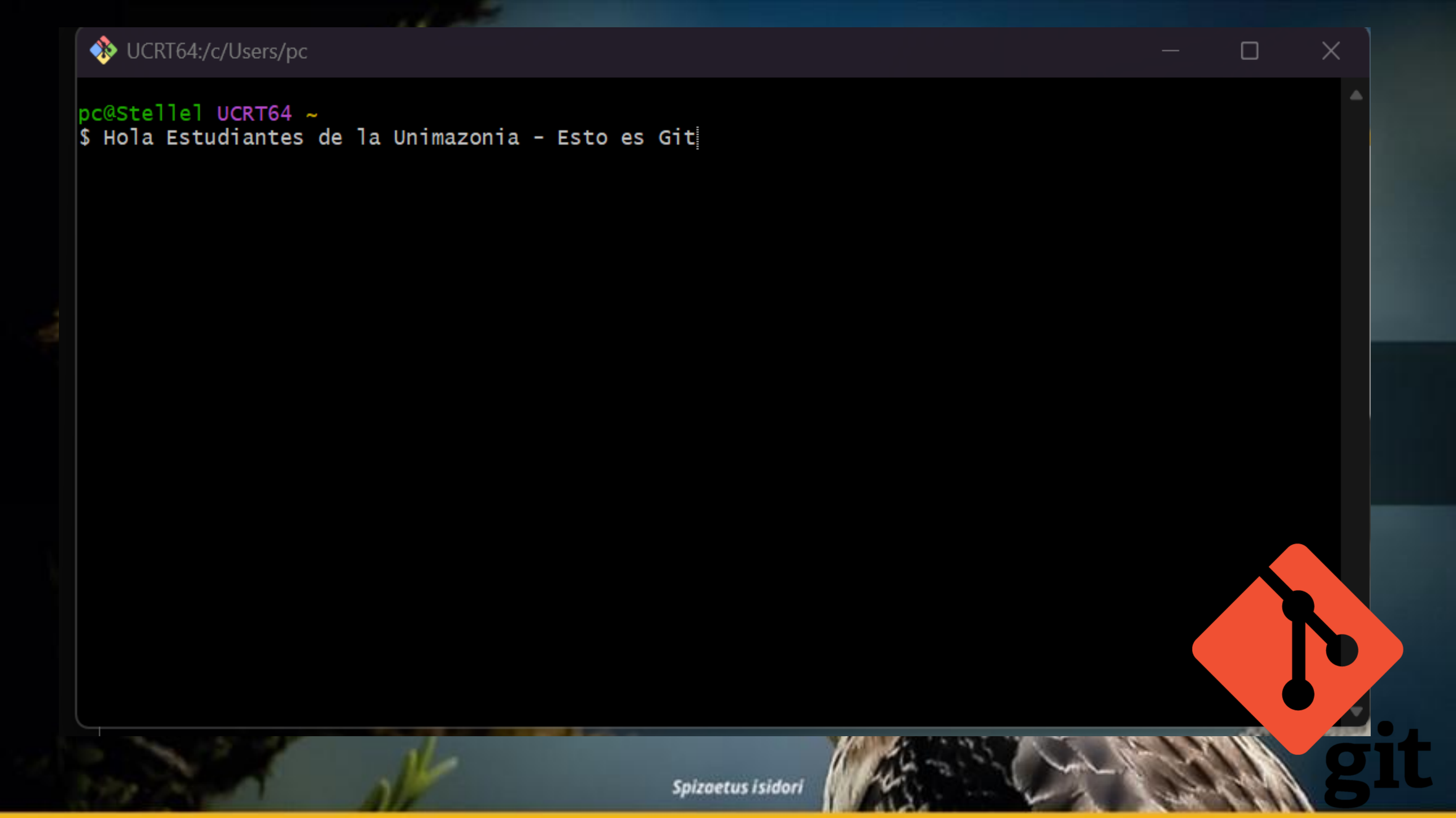


Git: Sistema Distribuido



Distribución total: Cada desarrollador posee una copia completa y local del historial. Permite trabajar **offline**, a **velocidad extrema** y con **tolerancia a fallos**.

Gemini Notebook


UCRT64:/c/Users/pc
pc@Stellel UCRT64 ~
$ Hola Estudiantes de la Unimazonia - Esto es Git
Spizaetus isidori
git

Dashboard

Type / to search

Home
Feed

Top repositories

Stellel-One/QueApp
Stellel-One/Mi-Primer-Robot
Stellel-One/TucanGo
Stellel-One/Better-Call-Pulido
Stellel-One/TucanTrace
Stellel-One/Stellel.com
Juan-Guillermo-Ferrer1/queseria-pradera
Stellel-One/queseria-pradera

Preview

Pull requests View all

No pull requests found, try a different filter.

Issues View all

No issues found, try a different filter.



UNIVERSE'26

Catch up on all that was announced.

See what's new

Latest from our changelog

16 hours ago
Webhook IP ranges added to the Meta API

2 days ago
Route and orchestrate models with GitHub Copilot Fusion

2 days ago
GitHub Copilot app, Universe releases

2 days ago
GitHub Enterprise now includes access to GitHub Copilot

View changelog →

© 2026 GitHub, Inc. Terms Privacy Security Status Community Docs Contact Manage cookies Do not share my personal information

Spizaetus isidori

GitHub

Matriz de Contraste: El Motor vs. La Red

	Git	GitHub
Naturaleza	Motor subyacente de control de versiones	Plataforma de alojamiento en la nube
Entorno	Sistema Local (Tu máquina)	Sistema Remoto (Servidor Global)
Interfaz	Interfaz de Línea de Comandos (CLI)	Interfaz Gráfica Web (GUI)
Propósito Core	Registra el historial y gestiona versiones	Fomenta colaboración (Pull Requests, Issues, CI/CD)

Gemini Notebook

Arquitectura Interna: Los 3 Estados de Git

Working Directory (Modificado)

Tu taller local. Los archivos tienen cambios, pero Git aún no los rastrea oficialmente.



`git add [archivo]`

Staging Area (Preparado)

La plataforma de carga. Se seleccionan archivos específicos para el próximo empaquetado.



`git commit -m 'mensaje'`

Repository (Confirmado)

La bóveda inmutable. Los cambios se guardan permanentemente en la base de datos local.



Gemini Notebook

20 COMANDOS DE GIT



1 git init

Crear repositorio desde cero.

```
$ mkdir app
$ cd app
$ git init
```

2 git clone

Clonar repo y revisar estado.

```
$ git clone repo
$ cd repo
$ git status
```

3 git add

Crear, agregar, confirmar archivo.

```
$ touch file.txt
$ git add file.txt
$ git commit -m "Add"
```

4 git branch

Crear y cambiar rama.

```
$ git branch dev
$ git checkout dev
$ git status
```

5 git checkout

Unir ramas y subir.

```
$ git checkout main
$ git merge dev
$ git push origin main
```

6 git pull

Bajar cambios y revisar.

```
$ git pull origin main
$ git status
$ git log
```

7 git diff

Modificar, ver, agregar.

```
$ echo "Hi" >> file.txt
$ git diff
$ git add file.txt
```

8 git reset

Deshacer cambios archivo.

```
$ git reset file.txt
$ git checkout -- file.txt
$ git status
```

9 git stash

Guardar, actualizar, recuperar.

```
$ git stash
$ git pull origin main
$ git stash pop
```

10 git fetch

Traer remoto y ver.

```
$ git fetch origin
$ git checkout -b new
$ git log
```

11 git tag

Crear y subir tag.

```
$ git tag v1.0
$ git push origin v1.0
$ git tag
```

12 git remote

Ver y agregar remoto.

```
$ git remote -v
$ git remote add u URL
$ git remote -v
```

13 git rm

Eliminar y subir archivo.

```
$ git rm file.txt
$ git commit -m "del"
$ git push
```

14 git rebase

Reaplicar y subir cambios.

```
$ git rebase main
$ git status
$ git push --force
```

15 git config

Configurar datos usuario.

```
$ git config --g name
$ git config --g mail
$ git config --list
```

16 git log

Ver historial compacto.

```
$ git log --oneline
$ git log --graph
$ git log --stat
```

17 git cherry-pick

Aplicar y ajustar commit.

```
$ git cherry-pick HASH
$ git commit --amend
$ git log
```

18 git diff

Comparar diferencias cambios.

```
$ git diff HEAD
$ git diff --staged
$ git diff b1 b2
```

19 git clean

Limpiar archivos sueltos.

```
$ git clean -n
$ git clean -f
$ git status
```

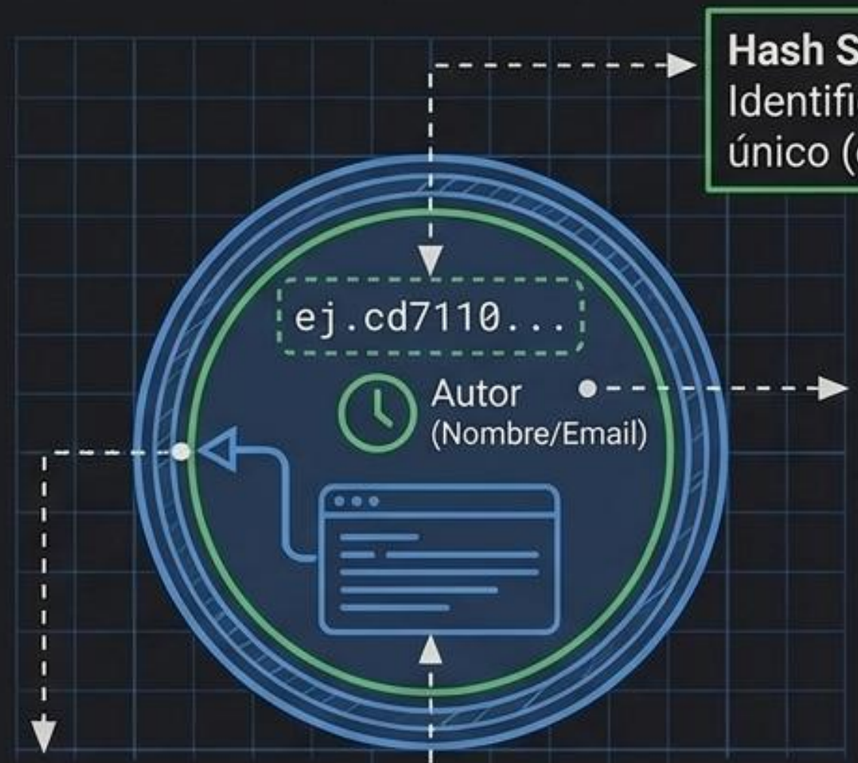
20 git bisect

Encontrar commit con error.

```
$ git bisect start
$ git bisect bad
$ git bisect good C
```



Anatomía de un Commit: La Máquina del Tiempo



Hash SHA-1:
Identificador criptográfico único (ej. `cd7110...`)

Metadatos: Autor (Nombre/Email) y Timestamp (Fecha/Hora)

Puntero Parent:
Referencia criptográfica al commit anterior

Mensaje: Descripción semántica del cambio

>_

`git status`

El radar en tiempo real de tu Working Directory.

`git log`

El registro histórico e inmutable de tus commits.

Gemini Notebook

Navegando y Deshaciendo el Espacio-Tiempo



Inspeccionar el pasado

>_

```
git checkout [hash]
```

Mueve el HEAD para visualizar código antiguo sin alterar la historia.

Destruir y Revertir

>_

```
git reset --hard
git revert
```

Reset elimina commits permanentemente. Revert crea un nuevo commit que deshace cambios de forma segura.

Gemini Notebook

Realidades Paralelas: Ramas (Branches)

El aislamiento funcional.

Múltiples ingenieros pueden modificar el mismo código base creando universos paralelos temporales que no afectan la versión de producción (main).




```
>_
```

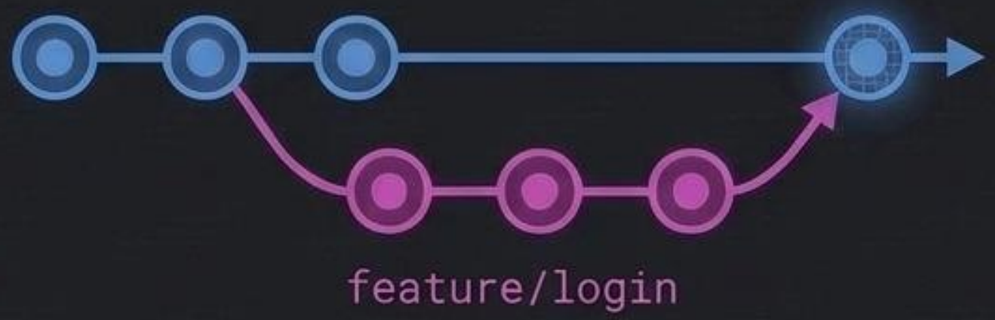
`git branch [nombre]` : Crea una nueva realidad paralela.

`git switch [nombre]` : Te transporta a esa rama específica.

Gemini Notebook

Convergencia: Fusiones (Merges) y Conflictos

Fusión Limpia (git merge)




```
>_
git merge feature/login
```

Integra la historia de una rama en otra de forma automática cuando no hay cambios superpuestos.

Colisión y Resolución



Git detiene la fusión. El ingeniero debe intervenir, editar y confirmar.

Gemini Notebook

Cruzando el Puente hacia la Nube

Repositorio Local (Git)



Repositorio Remoto (GitHub)

```
>_
git remote add origin https://github.com/usuario/repo.git
```

Origin: Es simplemente el alias estándar que Git le da a la URL de tu repositorio en la nube.

Para colaborar, el repositorio local debe vincularse a esta fuente única de la verdad.

Gemini Notebook

La Mecánica de Sincronización

`git clone`

De Nube a Local Nuevo.

Descarga un repositorio remoto completo por primera vez.



`git push`

De Local a Nube.

Empuja tus commits locales confirmados hacia el repositorio remoto.



`git fetch`

De Nube a Local (Solo Info)

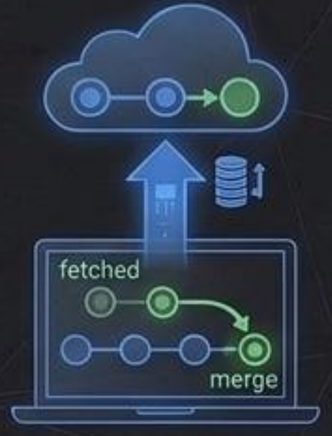
Descarga el historial de cambios, pero no los fusiona. Permite inspeccionar.



`git pull`

De Nube a Local (Integración).

Ejecuta un fetch seguido de un merge. Actualiza tu código con la nube.



Gemini Notebook

El Estándar de la Industria: Pull Requests (PR)

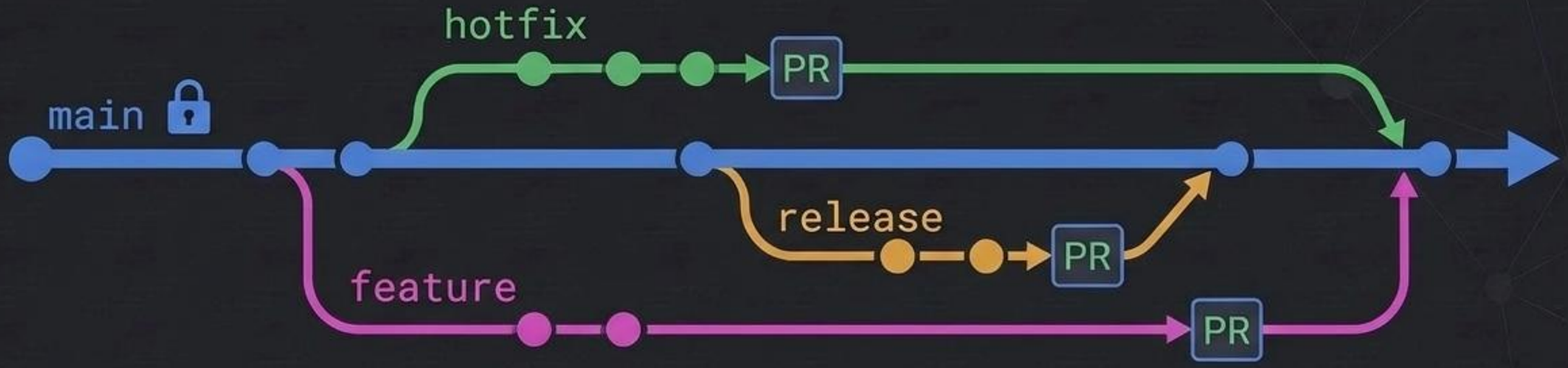
Un PR no es un comando de Git. Es una característica colaborativa de GitHub para solicitar integración.

- Revisión por Pares (Code Review): Búsqueda de ineficiencias antes de fusionar.
- Barrera de Calidad: Protege la rama main de código roto.
- Discusión Centralizada: Foro arquitectónico sobre la nueva funcionalidad.



Gemini Notebook

Flujos de Trabajo Colaborativos



REGLA DE ORO

La rama `main` es sagrada. Nunca se hace un commit directo a `main` en producción. Todo desarrollo ocurre en ramas secundarias y cruza hacia `main` únicamente tras la aprobación de un Pull Request.

Gemini Notebook

Reglas de Oro del Ingeniero de Software

[✅] Dominio del .gitignore

Excluir compilados, dependencias pesadas y credenciales. Nunca subir secretos a la nube.

[✅] Commits Atómicos

Un commit = una única unidad lógica de cambio, no un revoltijo de varias cosas.

[✅] Mensajes Descriptivos e Imperativos

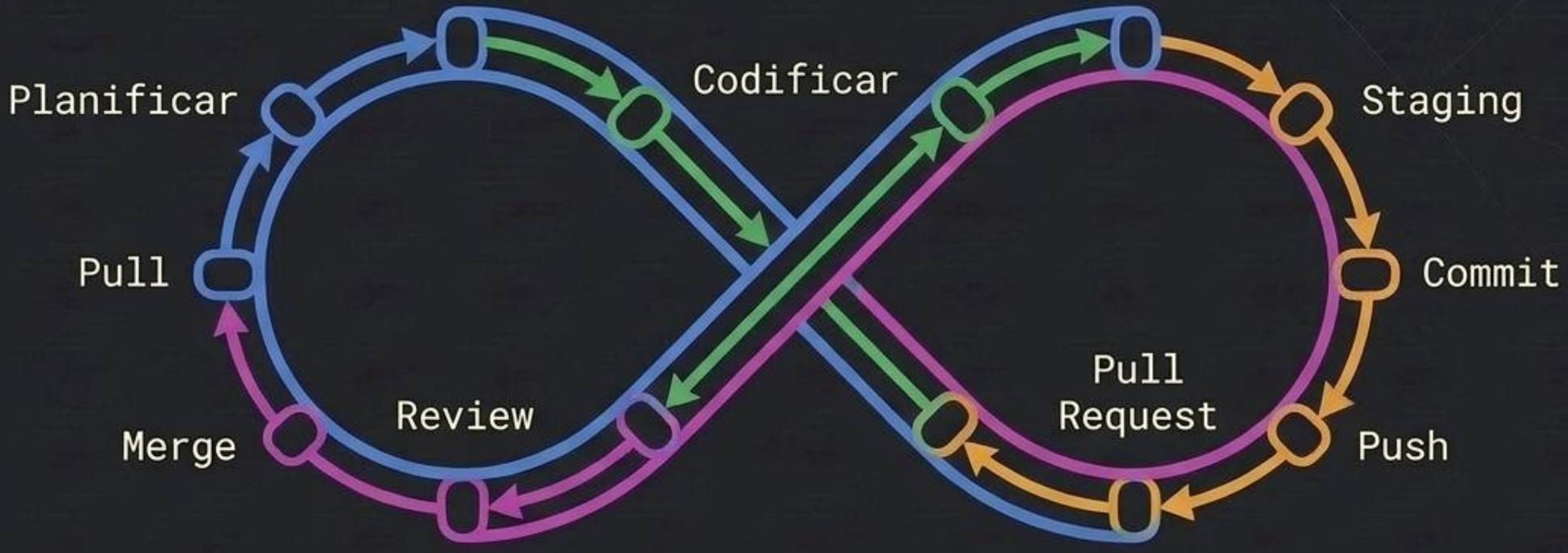
Escribir como órdenes al sistema (ej. 'Agrega validación de email', no 'cambios varios').

[✅] Sincronización Continua

Ejecutar git pull frecuentemente antes de codificar para evitar conflictos masivos.

Gemini Notebook

Síntesis: El Sistema Nervioso del CI/CD



Git y GitHub trascienden el almacenamiento. Son el tejido conectivo de la Integración Continua y el Despliegue Continuo (CI/CD). Dominar este flujo cíclico es dominar la coreografía del software a nivel empresarial.

Gemini Notebook